

# 程序设计Ⅶ

张宏斌 副教授

## 上周完成的任务

- 了解了课程的特点、学习的方法
- 掌握了编程的概念
- 安装了Python系统
- 安装了开发环境，推荐使用Geany、JupyterLab、TRAE
- 写了第一个Python程序
- 学会了用pip安装扩展库
  - 安装了NumPy、Pillow等库
- 尝试导入安装的扩展库，并进行照片处理
- 用TRAE做了一个小项目

# 课程微信群

- 要求大家都改为真实姓名，不然可能会被踢出群

## 第二章 Python基本操作

- 1 掌握计算思维、变量基本概念
- 2 熟练表达式基本操作，特别是运算符
- 3 掌握整数类型基本概念和操作
- 4 掌握浮点数类型基本概念和操作
- 5 掌握输入输出函数的基本用法
- 6 掌握布尔类型基本概念和操作
- 7 了解常用内置函数
- 8 掌握导入模块的方法

# 计算思维

- 计算思维是一种系统化的问题解决思维方式，它不是单纯指“会写代码”，而是一种逻辑严谨、结构化的思维方式，**把任何复杂问题拆解成计算机（或人）能一步步执行的清晰步骤的过程**
- 计算思维 = 用计算机的逻辑去思考现实问题
- 这种思维最早由卡内基梅隆大学教授Jeannette Wing（周以真）在2006年提出，现在已成为编程教育、人工智能、甚至日常决策的核心能力
- 计算思维通常被拆成四个相互关联的要素：**分解（Decomposition）、模式识别（Pattern Recognition）、抽象（Abstraction）、算法设计（Algorithm）、评估与优化**

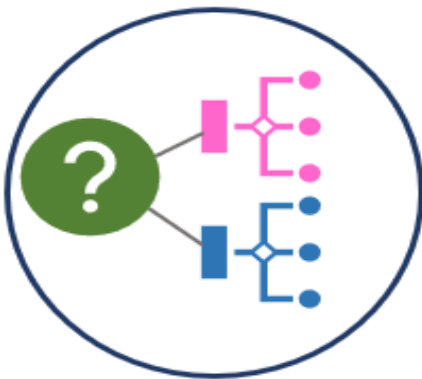


中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

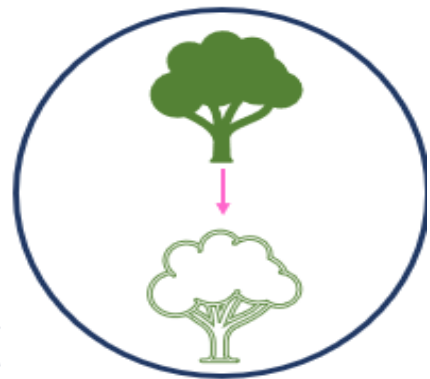
张宏斌 副教授 2026年春



## Decomposition



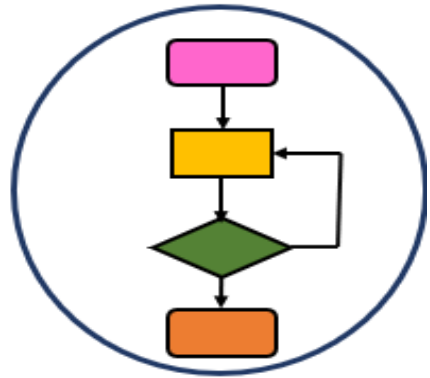
## Abstraction



## Computational Thinking



## Pattern recognition



## Algorithm

# 分解

- 把一个大问题拆分成若干个更小、更易处理的小问题
- 就像整理一个杂乱的房间，我们不会一次性打扫整个屋子，而是会分成整理书桌、收拾衣柜、扫地等小任务
- 例子：想“做一顿饭”
- 普通思维：直接去厨房做
- **计算思维**：分解成：
  - 子问题1：决定菜单
  - 子问题2：列购物清单
  - 子问题3：准备食材
  - 子问题4：烹饪顺序
  - 子问题5：摆盘上桌



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



# 模式识别

- 在多个问题中发现相似之处，从而复用已有的解决方案
- 比如我们学会了计算圆的面积，当遇到计算三角形面积时，虽然公式不同，但“用数学公式计算图形面积”这个模式是共通的
- 例子：你每天早上都要穿衣服
- 普通思维：每次都重新想“今天穿什么”
- **计算思维**：发现规律——“工作日穿正装、周末穿休闲、雨天带伞”。这就是模式！以后遇到“穿衣”问题，直接套用这个模式即可，不用每次从零开始思考



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



# 抽象

- 忽略无关的细节，只关注**核心信息**
- 如要设计一个地图应用，不需要画出每一棵树，只需要抽象出道路、建筑等关键元素
- 例子：设计一个“学生管理系统”
- 普通思维：把每个学生的身高、体重、爱好、父母职业全记下来
- **计算思维**：抽象出真正需要的信息——只记录“姓名、学号、成绩、班级”。其他细节（如爱好）暂时忽略，因为它们对“管理系统”这个核心目标没用。这就是抽象：**去掉噪声，抓住本质**

# 算法设计

- 把解决方案写成一系列清晰、可重复、有限步骤的指令。这些指令必须清晰、无歧义，并且最终能得出结果
- 比如做菜时，菜谱就是一套算法：先洗菜、再切菜、然后下锅炒
- 经典例子：给多个数排序（从小到大）
- 普通人可能会说“把小的放前面”
- **计算思维**的算法（冒泡排序简化版）：
  - 步骤1：比较第一个数和第二个数
  - 步骤2：如果第一个 > 第二个，就交换它们
  - 步骤3：重复步骤1-2，直到没有需要交换的为止



# 评估与优化

- 评估结果
- 优化过程（算法）
- 不断进行

# 计算（编程）思维的其他重要习惯

- **调试思维**：遇到问题不崩溃，先问“哪里出错了？”，而不是“我不行”。像程序员一样，逐行检查、打印中间结果
- **迭代思维**：先做一个能跑的最简版，然后不断改进。就像写代码，先写Hello World，再慢慢加功能
- **逻辑严谨**：不能有歧义。不能说“差不多就行”，必须说“如果A成立则执行B，否则执行C”

# 练习：如何用计算思维“规划一次旅行”

- 目标：去泰国旅行7天，用计算思维规划

## Python的程序结构

- 程序由模块组成

```
import math  
from PIL import Image  
test.py
```

- 模块包含语句

```
x = a ** 2 + b ** 2 + math.sqrt(128)
```

- 语句包含表达式

```
a ** 2
```

- 表达式创建并处理对象（对象由数据和行为两部分组成）

```
x, a, b, 2, math.sqrt(), 128, **, .....
```

# Python的对象

- 对象是python语言中最基本的概念，在python中处理的一切都是对象
- python中有许多内置对象可供编程者使用，内置对象可直接使用，如数字、字符串、列表、del、内置函数等
- 非内置对象需要导入模块才能使用，如正弦函数 $\sin(x)$ ，随机数产生函数`random()`等

## 变量与常量

- 在程序设计过程中，对于不需要改变且不能改变的字面值，称为常量
- 与常量相反，变量的值可以改变。Python中使用变量来收集和记录数据
- 每个变量用一个变量名来表示，在使用前都必须赋值。使用等号(=)来给变量赋值，变量名放在等号的左边，需要记录的数据放在等号的右边：

```
rate = 0.00125
```



# 变量命名规则

- 在定义变量名的时候，需要注意以下问题：
  - ✓ 变量命名规则是：（字母或下划线）+（任意数量的字母、数字或下划线），字母可以为中文，但不建议用中文
  - ✓ 变量名必须以字母或下划线开头，但以下划线开头的变量在Python中有特殊含义；
  - ✓ 变量名中不能有空格以及标点符号（括号、引号、逗号、斜线、反斜线、冒号、句号、问号等等）；
  - ✓ 不能使用关键字作变量名，可以导入keyword模块后使用`print(keyword.kwlist)`查看所有Python关键字；
  - ✓ 变量名对英文字母的大小写敏感，例如student和Student是不同的变量。
  - ✓ 不建议使用系统内置的模块名、类型名或函数名以及已导入的模块名及其成员名作变量名，这将会改变其类型和含义，可以通过`dir(__builtins__)`查看所有内置模块、类型和函数
- 建议变量名尽量表示变量的含义，慎用O（大写）和o（小写），student\_no好过s\_n



# Python的数字

- 在Python中，内置的数字类型有整数、实数（浮点数）和复数

`x = 125`

`y = 12.56`

`z = 3 + 4j`



# 整数

- **整数**是财经领域经常用到的数字类型。例如股票数量、交易天数、开户人数等
- **整数**用关键字 **int** 表示（即英文 integer 的简写），包括正整数、零和负整数
- 整型数全部由数字构成，不包括小数部分
- Python中的整型数据没有大小限制，支持任意大的数字
- 整型数可以用**4种进制**表示：二进制（0b...）、十进制、八进制（0o...）和十六进制（0x...）



# 实数（浮点数）

- **浮点型（即浮点数）**也是财经领域最常用的数据类型。如股票价格、利率、汇率等都是浮点型
- 在 Python 中，浮点型用关键字 **float** 表示，是由整数、小数点和小数构成的数字
- 使用**浮点数**的一些限制：
  - 浮点数可以表示非常大或非常小的数字，但是与整数不同，**Python 中的浮点数的取值范围存在限制**，这种限制与不同的计算机系统有关
  - 浮点数的最大精度是 16 位数字。这个限制对于绝大多数应用来说已经足够了
  - 当使用算术运算得到**浮点数**结果时，**有时计算并没有那么精确**

```
❶>>> 0.1 + 0.2
0.300000000000000040
❷>>> 0.1 + 0.2 == 0.3
False
```



# 数字类型的转换

- **隐式转换：**
  - 两个整数运算，结果为整数（除法运算“/”除外，其运算结果为小数）
  - 两个浮点数运算，结果为浮点数
  - 整数和浮点数运算，结果为浮点数
- **显式转换：**
  - **int(x)**，将 x 转换为整数，x 可以是浮点数或字符串
  - **float(x)**，将 x 转换为浮点数，x 可以是整数或字符串

```
>>> int(10.8)
10
>>> float(10)
10.0
```



# Python的运算符与表达式

- Python是面向对象的编程语言，在Python中一切都是对象。对象由数据和行为两部分组成。运算符是表现对象行为的一种主要形式，不同的对象支持的运算符有所不同，同一种运算符作用于不同的对象时也可能表现出不同的行为，这是Python运算符“多态”的体现
- 在Python中，单个常量或变量可以看作最简单的表达式，使用除赋值运算符之外的其他任意运算符和函数调用连接的式子也属于表达式



# Python的常用运算符

| 运算符                 | 功能说明                                             |
|---------------------|--------------------------------------------------|
| +                   | 算术 <b>加法</b> ，列表、元组、字符串 <b>合并与连接</b> ， <b>正号</b> |
| -                   | 算术 <b>减法</b> ，集合 <b>差集</b> ， <b>相反数</b>          |
| *                   | 算术 <b>乘法</b> ，序列 <b>重复</b>                       |
| /                   | <b>真除法</b>                                       |
| //                  | 求 <b>整商</b> ，但如果操作数中有实数的话，结果为实数形式的整数             |
| %                   | 求 <b>余数</b> ，字符串 <b>格式化</b>                      |
| **                  | <b>幂运算</b>                                       |
| <、<=、>、>=、<br>==、!= | (值) <b>大小比较</b> ，集合的包含关系比较                       |

- 算术运算符遵循“**先幂，再乘除，后加减**”的基本运算原则
- 强烈建议在编写复杂表达式时使用**圆括号**来明确说明其中的逻辑来提高代码可读性



# Python高级赋值运算符

- 在Python中，“**=**”是基本**赋值**运算符，还有：

| Python 赋值运算符 |                   |       |                         |
|--------------|-------------------|-------|-------------------------|
| 运算符          | 举例                | x 的结果 | 功能说明                    |
| +=           | x = 10<br>x += 8  | 18    | 加法赋值运算符，等价于 x = x + 8   |
| -=           | x = 10<br>x -= 8  | 2     | 减法赋值运算符，等价于 x = x - 8   |
| *=           | x = 10<br>x *= 8  | 80    | 乘法赋值运算符，等价于 x = x * 8   |
| /=           | x = 10<br>x /= 8  | 1.25  | 除法赋值运算符，等价于 x = x / 8   |
| //=          | x = 10<br>x //= 8 | 1     | 整除赋值运算符，等价于 x = x // 8  |
| %=           | x = 10<br>x %= 8  | 2     | 取余数赋值运算符，等价于 x = x % 8  |
| **=          | x = 3<br>x **= 2  | 9     | 幂运算赋值运算符，等价于 x = x ** 2 |





# 求模运算符%

- **求模运算符%**是一个很有用的工具，它将两个数相除并返回余数

```
number = input("Enter a number, and I'll tell you if it's even or odd:\n")
number = int(number)
```

```
if number % 2 == 0:
    print("{0} is even.".format(number))
else:
    print("{0} is odd.".format(number))
```

# 别忘了冒号:  
# 注意缩进

# 布尔类型

- **布尔类型**只有两种值：**True** 和 **False**
- **布尔值 True 和 False** 不像字符串，**两边没有引号**，它们总是以大写字母 T 或 F 开头，后面的字母小写

```
>>> 1 < 2
True
>>> 1 > 2
False
```



# 逻辑运算符

- 在 Python 中有三种逻辑运算：**and**（且）、**or**（或）和 **not**（否）
- **and** 和 **or** 操作符总是接受两个布尔值（或表达式），所以它们被称为“二元”操作符
- 对于 **and** 操作符，如果两个布尔值都为 **True**，**and** 操作符就将表达式求值为 **True**，否则求值为 **False**

```
>>> True and True
True
>>> True and False
False
```

## 逻辑运算符（续）

- 对于 **or** 操作符，只要有一个布尔值为真，**or** 操作符就将表达式求值为 **True**。如果都是 **False**，所求值为 **False**

```
>>> False or True
True
>>> False or False
False
```

- **not** 操作符只作用于一个布尔值（或表达式）。求值为相反的布尔值

```
>>> not True
False
>>> not not False
False
```

# 比较运算符

- 布尔类型值通常来自比较运算的结果

【例】股价波动率为 0.25，判断其是否符合投资者股价波动率低于 20% 的限制条件

```
>>> volatility = 0.25
>>> volatility < 0.2
False
```

- Python表达式支持多个比较运算，这种表达式和数学中的表达方式非常接近

【例】两个数的平均数总是在这两个数的较小数和较大数之间

```
>>> x = 10
>>> y = 8
>>> min(x, y) <= (x + y) / 2 <= max(x, y)
True
```



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



# 比较运算符

- 相等 ==
- 不等于 !=
- 小于 <
- 不大于 <=
- 大于 >
- 不小于 >=



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



# Python常用的内置函数

- Python有许多内置函数，详见  
<https://docs.python.org/3/library/functions.html>

| Python 中常用内置函数 |                         |         |                                                                   |
|----------------|-------------------------|---------|-------------------------------------------------------------------|
| 函数名称           | 代码示例                    | 结果      | 功能描述                                                              |
| abs            | x = -10<br>abs(x)       | 10      | 求绝对值                                                              |
| float          | float('10.8')           | 10.8    | 将整数或者字符串转换为浮点数                                                    |
| int            | int(10.8)               | 10      | 将浮点数截断小数部分后转换为整数或者将只包含数字的字符串转换为整数                                 |
| len            | len('金融科技')             | 4       | 获得字符串、列表、元组、集合或字典的长度（即元素个数）                                       |
| max            | max(10,-10.8,3,58,20.5) | 58      | 获得一组数据中最大值                                                        |
| min            | min(10,-10.8,3,58,20.5) | -10.8   | 获得一组数据中最小值                                                        |
| range          | list(range(0,6,2))      | [0,2,4] | 产生一个可迭代对象，在此列中从 0 开始到 6 之前（即不包括 6），以步长为 2 产生可迭代对象后利用 list 函数转换成列表 |
| sum            | sum([10,3,-10.8,58])    | 60.2    | 获得一组数据的和，在此列中获得列表[10, 3, -10.8, 58]中的数据之和                         |



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



## Built-in Functions

### A

[abs\(\)](#)  
[aiter\(\)](#)  
[all\(\)](#)  
[anext\(\)](#)  
[any\(\)](#)  
[ascii\(\)](#)

### B

[bin\(\)](#)  
[bool\(\)](#)  
[breakpoint\(\)](#)  
[bytearray\(\)](#)  
[bytes\(\)](#)

### C

[callable\(\)](#)  
[chr\(\)](#)  
[classmethod\(\)](#)  
[compile\(\)](#)  
[complex\(\)](#)

### D

[delattr\(\)](#)  
[dict\(\)](#)  
[dir\(\)](#)  
[divmod\(\)](#)

### E

[enumerate\(\)](#)  
[eval\(\)](#)  
[exec\(\)](#)

### F

[filter\(\)](#)  
[float\(\)](#)  
[format\(\)](#)  
[frozenset\(\)](#)

### G

[getattr\(\)](#)  
[globals\(\)](#)

### H

[hasattr\(\)](#)  
[hash\(\)](#)  
[help\(\)](#)  
[hex\(\)](#)

### I

[id\(\)](#)  
[input\(\)](#)  
[int\(\)](#)  
[isinstance\(\)](#)  
[issubclass\(\)](#)  
[iter\(\)](#)

### L

[len\(\)](#)  
[list\(\)](#)  
[locals\(\)](#)

### M

[map\(\)](#)  
[max\(\)](#)  
[memoryview\(\)](#)  
[min\(\)](#)

### N

[next\(\)](#)

### O

[object\(\)](#)  
[oct\(\)](#)  
[open\(\)](#)  
[ord\(\)](#)

### P

[pow\(\)](#)  
[print\(\)](#)  
[property\(\)](#)

### R

[range\(\)](#)  
[repr\(\)](#)  
[reversed\(\)](#)  
[round\(\)](#)

### S

[set\(\)](#)  
[setattr\(\)](#)  
[slice\(\)](#)  
[sorted\(\)](#)  
[staticmethod\(\)](#)  
[str\(\)](#)  
[sum\(\)](#)  
[super\(\)](#)

### T

[tuple\(\)](#)  
[type\(\)](#)

### V

[vars\(\)](#)

### Z

[zip\(\)](#)

[\\_\\_import\\_\\_\(\)](#)

## Python常见内置函数（续）

| 函数                           | 功能简要说明                                                                     |
|------------------------------|----------------------------------------------------------------------------|
| <b>abs(x)</b>                | 返回数字x的 <b>绝对值</b> 或复数x的 <b>模</b>                                           |
| <b>all(iterable)</b>         | 如果对于可迭代对象中所有元素x都等价于True，也就是对于所有元素x都有bool(x)等于True，则返回True。对于空的可迭代对象也返回True |
| <b>any(iterable)</b>         | 只要可迭代对象iterable中存在元素x使得bool(x)为True，则返回True。对于空的可迭代对象，返回False              |
| <b>ascii(obj)</b>            | 把对象转换为ASCII码表示形式，必要的时候使用转义字符来表示特定的字符                                       |
| <b>bin(x)</b>                | 把整数x转换为二进制串表示形式                                                            |
| <b>bool(x)</b>               | 返回与x等价的布尔值True或False                                                       |
| <b>bytes(x)</b>              | 生成字节串，或把指定对象x转换为字节串表示形式                                                    |
| <b>callable(obj)</b>         | 测试对象obj是否可调用。类和函数是可调用的，包含__call__()方法的类的对象也是可调用的                           |
| <b>compile()</b>             | 用于把Python代码编译成可被exec()或eval()函数执行的代码对象                                     |
| <b>complex(real, [imag])</b> | 返回复数                                                                       |
| <b>chr(x)</b>                | 返回Unicode编码为x的字符                                                           |



## Python常见内置函数（续）

| 函数                                   | 功能简要说明                                                                                  |
|--------------------------------------|-----------------------------------------------------------------------------------------|
| <b>delattr(obj, name)</b>            | 删除属性，等价于del obj.name                                                                    |
| <b>dir(obj)</b>                      | 返回指定对象或模块obj的成员列表，如果不带参数则返回当前作用域内所有标识符                                                  |
| <b>divmod(x, y)</b>                  | 返回包含整商和余数的元组((x-x%y)/y, x%y)                                                            |
| <b>enumerate(iterable[, start])</b>  | 返回包含元素形式为(0, iterable[0]), (1, iterable[1]), (2, iterable[2]), ...的迭代器对象                |
| <b>eval(s[, globals[, locals]])</b>  | 计算并返回字符串s中表达式的值                                                                         |
| <b>exec(x)</b>                       | 执行代码或代码对象x                                                                              |
| <b>exit()</b>                        | 退出当前解释器环境                                                                               |
| <b>filter(func, seq)</b>             | 返回filter对象，其中包含序列seq中使得单参数函数func返回值为True的那些元素，如果函数func为None则返回包含seq中等价于True的元素的filter对象 |
| <b>float(x)</b>                      | 把整数或字符串x转换为浮点数并返回                                                                       |
| <b>frozenset([x])</b>                | 创建不可变的集合对象                                                                              |
| <b>getattr(obj, name[, default])</b> | 获取对象中指定属性的值，等价于obj.name，如果不存在指定属性则返回default的值，如果要访问的属性不存在并且没有指定default则抛出异常             |





## Python常见内置函数（续）

| 函数                                                   | 功能简要说明                                                                    |
|------------------------------------------------------|---------------------------------------------------------------------------|
| <code>globals()</code>                               | 返回包含当前作用域内全局变量及其值的字典                                                      |
| <code>hasattr(obj, name)</code>                      | 测试对象obj是否具有名为name的成员                                                      |
| <code>hash(x)</code>                                 | 返回对象x的哈希值，如果x不可哈希则抛出异常                                                    |
| <code>help(obj)</code>                               | 返回对象obj的帮助信息                                                              |
| <code>hex(x)</code>                                  | 把整数x转换为十六进制串                                                              |
| <code>id(obj)</code>                                 | 返回对象obj的标识（内存地址）                                                          |
| <code>input([提示])</code>                             | 显示提示，接收键盘输入的内容，返回字符串                                                      |
| <code>int(x[, d])</code>                             | 返回实数（float）、分数（Fraction）或高精度实数（Decimal）x的整数部分，或把d进制的字符串x转换为十进制并返回，d默认为十进制 |
| <code>isinstance(obj, class-or-type-or-tuple)</code> | 测试对象obj是否属于指定类型（如果有多个类型的话需要放到元组中）的实例                                      |
| <code>iter(...)</code>                               | 返回指定对象的可迭代对象                                                              |
| <code>len(obj)</code>                                | 返回对象obj包含的元素个数，适用于列表、元组、集合、字典、字符串以及range对象和其他可迭代对象                        |

## Python常见内置函数（续）

| 函数                                                                                                | 功能简要说明                                                 |
|---------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <code>list([x])</code> 、 <code>set([x])</code> 、 <code>tuple([x])</code> 、 <code>dict([x])</code> | 把对象x转换为列表、集合、元组或字典并返回，或生成空列表、空集合、空元组、空字典               |
| <code>locals()</code>                                                                             | 返回包含当前作用域内局部变量及其值的字典                                   |
| <code>map(func, *iterables)</code>                                                                | 返回包含若干函数值的map对象，函数func的参数分别来自于iterables指定的每个迭代对象，      |
| <code>max(x)</code> 、 <code>min(x)</code>                                                         | 返回可迭代对象x中的最大值、最小值，要求x中的所有元素之间可比较大小，允许指定排序规则和x为空时返回的默认值 |
| <code>next(iterator[, default])</code>                                                            | 返回可迭代对象x中的下一个元素，允许指定迭代结束之后继续迭代时返回的默认值                  |
| <code>oct(x)</code>                                                                               | 把整数x转换为八进制串                                            |
| <code>open(name[, mode])</code>                                                                   | 以指定模式mode打开文件name并返回文件对象                               |
| <code>ord(x)</code>                                                                               | 返回1个字符x的Unicode编码                                      |
| <code>pow(x, y, z=None)</code>                                                                    | 返回x的y次方，等价于 $x ** y$ 或 $(x ** y) \% z$                 |



## Python常见内置函数（续）

| 函数                                                                         | 功能简要说明                                                                                                      |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>print</b> (value, ..., sep=' ', end='\n', file=sys.stdout, flush=False) | 基本输出函数                                                                                                      |
| <b>quit</b> ()                                                             | 退出当前解释器环境                                                                                                   |
| <b>range</b> ([start,] end [, step] )                                      | 返回range对象，其中包含左闭右开区间[start,end)内以step为步长的整数                                                                 |
| <b>reduce</b> (func, sequence[, initial])                                  | 将双参数的函数func以迭代的方式从左到右依次应用至序列seq中每个元素，最终返回单个值作为结果。在Python 2.x中该函数为内置函数，在Python 3.x中需从functools中导入reduce函数再使用 |
| <b>repr</b> (obj)                                                          | 返回对象obj的规范化字符串表示形式，对于大多数对象有eval(repr(obj))==obj                                                             |
| <b>reversed</b> (seq)                                                      | 返回seq（可以是列表、元组、字符串、range以及其他可迭代对象）中所有元素逆序后的迭代器对象                                                            |



## Python常见内置函数（续）

| 函数                                                | 功能简要说明                                                                                  |
|---------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>round</b> (x [, 小数位数])                         | 对x进行四舍五入，若不指定小数位数，则返回整数                                                                 |
| <b>sorted</b> (iterable, key=None, reverse=False) | 返回排序后的列表，其中iterable表示要排序的序列或迭代对象，key用来指定排序规则或依据，reverse用来指定升序或降序。该函数不改变iterable内任何元素的顺序 |
| <b>str</b> (obj)                                  | 把对象obj直接转换为字符串                                                                          |
| <b>sum</b> (x, start=0)                           | 返回序列x中所有元素之和，返回start+sum(x)                                                             |
| <b>type</b> (obj)                                 | 返回对象obj的类型                                                                              |
| <b>zip</b> (seq1 [, seq2 [...]])                  | 返回zip对象，其中元素为(seq1[i], seq2[i], ...)形式的元组，最终结果中包含的元素个数取决于所有参数序列或可迭代对象中最短的那个             |



# 打印输出函数print()

- print函数的作用是将输入的参数打印出来，默认打印到屏幕上，一般用法如下：

```
print(<逗号分隔的需打印参数列表>)
```

【例】输出股票的代码、名称、开盘价、收盘价及收盘开盘价差。

```
>>> open_price, close_price = 35.2, 36.2
>>> print(600000, '浦发银行', open_price, close_price, close_price - open_price)
600000 浦发银行 35.2 36.2 1.0
```



## print()函数的参数

- print()函数的语法格式为：

**print(value1, value2, ..., sep=' ', end='\n', file=sys.stdout, flush=False)**

- **sep**参数之前为需要输出的内容（可以有多个）；
- **sep**参数用于指定数据之间的分隔符，默认为空格；
- **end**参数用于指定输出完数据之后再输出什么字符，默认为换行\n；
- **file**参数用于指定输出位置，默认为标准控制台，也可以重定向输出到文件

```
>>> print(1, 3, 5, 7, sep='\t') # 修改默认分隔符
1      3      5      7
>>> for i in range(10): # 修改end参数，每个输出之后不换行
    print(i, end='')
0 1 2 3 4 5 6 7 8 9
>>> with open('test.txt', 'a+') as fp:
    print('Hello world!', file=fp) # 重定向，将内容输出到文件中
```



# 输入函数input()

- **input()**函数用来接收用户的键盘输入，不论用户输入什么内容，**input()**一律返回字符串，必要的时候可以使用内置函数**int()**、**float()**或**eval()**对用户输入的内容进行类型转换，一般用法为：

```
input(<提示信息>)
```

注意参数为  
提示信息

【例】用户分别输入商品的价格和购买数量，计算需要支付的金额

```
>>> price = float(input("请输入商品价格 :"))
请输入商品价格 :10.8
>>> quantity = int(input("请输入购买数量 :"))
请输入购买数量 :100
>>> print('共需支付: {:.2f}元'.format(price * quantity))
共需支付:1080.00元
```



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



## Python常用内置数学函数

| 函数                                                                             | 返回值          |
|--------------------------------------------------------------------------------|--------------|
| abs(x)                                                                         | x的绝对值        |
| round(x)                                                                       | 把浮点数x四舍五入为整数 |
| round(x, n)                                                                    | 保留n位小数       |
| max((x <sub>1</sub> , x <sub>2</sub> , x <sub>3</sub> , ..., x <sub>n</sub> )) | 最大值          |
| min((x <sub>1</sub> , x <sub>2</sub> , x <sub>3</sub> , ..., x <sub>n</sub> )) | 最小值          |
| pow(x, y, z=None)                                                              | 求x的y次方       |



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



# 导入模块

- Python 本身自带了一些常用的模块（**标准库**），例如，**math**、**random**和**sys**等。这些模块不需要安装，但是在使用前需要导入
- 其他非标准库，又称为第三方库（**扩展库**），在首次使用前需要安装。例如，数据处理 常常用到的 **numpy**、**pandas**、**scipy** 和 **matplotlib** 等。扩展库使用时需要导入，如：

```
import numpy as np
```



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



AACSB  
ACCREDITED



ASSOCIATION  
AMBA  
ACCREDITED

## 导入模块（续）

- 用语句“**import 模块名**”直接导入整个模块：

```
>>> import math
>>> math.sqrt(4)
2.0
```

- 用语句“**import 模块名 as 名称**”导入整个模块的同时给该模块取个较短的**别名**：

```
>>> import numpy as np
>>> np.sqrt(4)
2.0
```



中山大學嶺南學院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



AACSB  
ACCREDITED



ASSOCIATION  
AMBA  
ACCREDITED

## 导入模块（续）

- 用语句“**import 模块名 . 子模块名 as 名称缩写**”导入**某个模块子模块**的同时给该子模块取个较短的别名：

```
>>> import matplotlib.pyplot as plt
>>> plt.title('人口数量(亿)')
Text(0.5, 1.0, '人口数量(亿)')
```

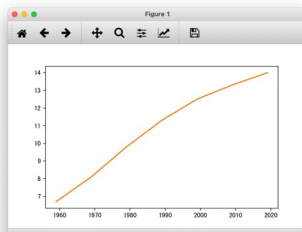
- 用语句“**from 模块名 import 函数**”导入模块中指定函数：

```
>>> from math import sqrt, exp
>>> sqrt(4)
2.0
```

## 导入模块（续）

- 用语句“**from 模块名 . 子模块名 import 函数**”导入**某个模块的子模块**中指定函数：

```
>>> from matplotlib.pyplot import plot, show
>>> plot([1959, 1969, 1979, 1989, 1999, 2009, 2019],
>>>      [6.7, 8.1, 9.8, 11.3, 12.5, 13.3, 14.0])
>>> show()
```





# math标准库中的常用函数

| 数学函数                      | 说明                  |
|---------------------------|---------------------|
| <code>exp(x)</code>       | 返回 $e$ 的 $x$ 的幂     |
| <code>fabs(x)</code>      | 返回 $x$ 的绝对值         |
| <code>factorial(x)</code> | 返回 $x$ 的阶乘          |
| <code>log(x)</code>       | 返回以 $e$ 为底的自然对数     |
| <code>log(x, y)</code>    | 返回以 $y$ 为底的 $x$ 的对数 |
| <code>log10(x)</code>     | 返回以 10 为底的 $x$ 的对数  |
| <code>pow(x, y)</code>    | 返回 $x$ 的 $y$ 次方     |
| <code>sqrt(x)</code>      | 返回 $x$ 的平方根         |
| 三角函数                      | 说明                  |
| <code>cos(x)</code>       | 返回 $x$ 弧度的余弦值       |
| <code>radians(x)</code>   | 将角 $x$ 从角度转换成弧度     |
| <code>degrees(x)</code>   | 将角 $x$ 从弧度转换成角度     |
| <code>sin(x)</code>       | 返回 $x$ 弧度的正弦值       |
| <code>tan(x)</code>       | 返回 $x$ 弧度的正切值       |

```
import math
print(math.log(100, 10))
```



中山大学岭南学院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春



## 总结

- Python的程序结构
- 变量的命名
- Python的数字类型：整数、浮点数、复数
- 算术运算符：+ - \* / // % \*\*
- 布尔值
- 逻辑运算符：and or not
- 比较运算符：> < >= <= == !=
- Python的常用内置函数
- `print()`和`input()`的用法
- 模块的导入方法
- `math`库中的常用数学函数



中山大学岭南学院  
LINGNAN COLLEGE SUN YAT-SEN UNIVERSITY

张宏斌 副教授 2026年春

